Assignment-21.3

2303A51304

Batch-05

Task-01:

Prompt:

Create a basic Python Expense Tracker application that records daily expenses with date, category, description, and amount using a menu-driven interface.

Then suggest improvements including:

Monthly expense summary, Input validation, Refactoring code into functions

Provide an improved version of the program with those enhancements.

Finally, compare the initial and improved versions to explain how AI assisted in collaborative coding.

Give complete runnable code and concise explanations.

Code:

```python
def add_expense():
    """Prompts the user for expense details and adds them to the CSV file."""
    print("\n--- Add New Expense ---")
    date_str = input("Enter date (YYYY-MM-DD): ")
    category = input("Enter category (e.g., Food, Transport, Utilities): ")
    description = input("Enter description: ")
    amount = input("Enter amount: ")

    # Basic validation (will be improved later)
    try:
        datetime.strptime(date_str, '%Y-%m-%d') # Check date format
        amount_float = float(amount) # Check amount is numeric
    except ValueError:
        print("Invalid date format or amount. Please try again.")
        return

    with open(EXPENSE_FILE, 'a', newline='') as file:
        writer = csv.writer(file)
        writer.writerow([date_str, category, description, amount_float])
    print("Expense added successfully!")

def view_expenses():
    """Reads and displays all recorded expenses from the CSV file."""
    print("\n--- All Expenses ---")
    if not os.path.exists(EXPENSE_FILE) or os.stat(EXPENSE_FILE).st_size == 0:
        print("No expenses recorded yet.")
        return

    with open(EXPENSE_FILE, 'r', newline='') as file:
```

```python
            print(f"{header[0]:<12} {header[1]:<15} {header[2]:<30} {header[3]:<1
            print("-" * 70)
        for row in reader:
            if row:
                print(f"{row[0]:<12} {row[1]:<15} {row[2]:<30} {float(row[3]):<10.2f}")
    print("\n")

def main():
    """Main function to run the expense tracker application."""
    initialize_expense_file()
    while True:
        print("\n--- Expense Tracker Menu ---")
        print("1. Add expense")
        print("2. View all expenses")
        print("3. Exit")
        choice = input("Enter your choice: ")

        if choice == '1':
            add_expense()
        elif choice == '2':
            view_expenses()
        elif choice == '3':
            print("Exiting Expense Tracker. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

# Run the application
main()
```

Output:

```
--- Expense Tracker Menu ---
1. Add expense
2. View all expenses
3. Exit
Enter your choice: 1

--- Add New Expense ---
Enter date (YYYY-MM-DD): 2026-04-01
Enter category (e.g., Food, Transport, Utilities): Food
Enter description: bought some food
Enter amount: 2000
Expense added successfully!

--- Expense Tracker Menu ---
1. Add expense
2. View all expenses
3. Exit
Enter your choice: 1

--- Add New Expense ---
Enter date (YYYY-MM-DD): 2026-03-12
Enter category (e.g., Food, Transport, Utilities): Transport
Enter description: Travelled
Enter amount: 5000
Expense added successfully!

--- Expense Tracker Menu ---
1. Add expense
2. View all expenses
3. Exit
Enter your choice: 2
```

```
Enter category (e.g., Food, Transport, Utilities): Transport
Enter description: Travelled
Enter amount: 5000
Expense added successfully!

--- Expense Tracker Menu ---
1. Add expense
2. View all expenses
3. Exit
Enter your choice: 2

--- All Expenses ---
Date         Category        Description                    Amount
----------------------------------------------------------------------
2026-04-01   Food            bought some food               2000.00
2026-03-12   Transport       Travelled                      5000.00



--- Expense Tracker Menu ---
1. Add expense
2. View all expenses
3. Exit
Enter your choice: 3
Exiting Expense Tracker. Goodbye!
```

Explanation:

The application is a command-line Expense Tracker used to record and manage daily spending.
Users can add expenses by entering details like date, category, description, and amount, and the data is saved for future use.
It allows viewing all recorded expenses in an organized format.
The enhanced version provides monthly summaries and category-wise expense analysis to understand spending patterns.
The program also includes input validation and a cleaner modular code structure for better reliability and maintainability.

Task-02:

Prompt:

Guide me in managing a Student Grade Calculator project using Git.

Initialize a Git repository for the project.

Create a new feature branch to add:

Grade classification (A, B, C, etc.), Average and percentage calculation

Show meaningful Git commit messages for each change.

Merge the feature branch back into the main branch and demonstrate how to resolve merge conflicts if they occur.

Provide commands, explanations, and example code where needed.

Code:

```
Provide commands, explanations, and example code where needed.'''
# Student Grade Calculator

def calculate_average(marks):
    return sum(marks) / len(marks)


def calculate_percentage(avg):
    return avg  # assuming marks are already out of 100

def classify_grade(percentage):
    if percentage >= 90:
        return "A"
    elif percentage >= 75:
        return "B"
    elif percentage >= 60:
        return "C"
    elif percentage >= 50:
        return "D"
    else:
        return "F"

def main():
    print("=== Student Grade Calculator ===")

    name = input("Enter student name: ")
    n = int(input("Enter number of subjects: "))

    marks = []

    for i in range(n):
        mark = float(input(f"Enter marks for subject {i+1}: "))
        marks.append(mark)

    avg = calculate_average(marks)
    percentage = calculate_percentage(avg)
```

```python
def classify_grade(percentage):
        return "F"

def main():
    print("=== Student Grade Calculator ===")

    name = input("Enter student name: ")
    n = int(input("Enter number of subjects: "))

    marks = []

    for i in range(n):
        mark = float(input(f"Enter marks for subject {i+1}: "))
        marks.append(mark)

    avg = calculate_average(marks)
    percentage = calculate_percentage(avg)
    grade = classify_grade(percentage)

    print("\n--- Result ---")
    print("Student:", name)
    print("Average:", round(avg, 2))
    print("Percentage:", round(percentage, 2), "%")
    print("Grade:", grade)


if __name__ == "__main__":
    main()
```

Output:

```
C:\AI_Assign>git init
Initialized empty Git repository in C:/AI_Assign/.git/

C:\AI_Assign>git add .

C:\AI_Assign>git commit -m "Initial commit"
On branch master

Initial commit
```

```
al/Python/pythoncore-3.14-64/python.exe c:/Users/po
_coding/assignment-21/task-02.py
=== Student Grade Calculator ===
Enter student name: Poojitha
Enter number of subjects: 3
Enter marks for subject 1: 50
Enter marks for subject 2: 70
Enter marks for subject 3: 90

--- Result ---
Student: Poojitha
Average: 70.0
Percentage: 70.0 %
Grade: C
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_co
```

Explanation:

This project is a Student Grade Calculator managed using Git version control to demonstrate how software development is organized professionally.

The application takes a student's marks as input, calculates the average and percentage, and assigns a grade (A, B, C, etc.) based on performance.
Git is used to track changes by first creating a repository and saving the initial version of the program.
New features like grade classification and calculations are developed

in a separate branch so the main code remains stable.
Finally, the feature branch is merged back into the main branch, showing how developers integrate updates and resolve conflicts during collaborative development.

Task-03:

Prompt:

working on a Password Validation module.

Assume the code was updated to include:

Minimum password length validation, Special character requirement

Generate clear and professional Git commit messages describing these changes.

Provide multiple commit message styles:

Short conventional commit, Detailed professional commit, Beginner/human-written style

Compare the AI-generated commit messages with human-written ones and briefly analyze clarity and effectiveness.

Keep responses concise and practical

Code:

```python
import re


def validate_password(password):
    errors = []

    # Rule 1: Minimum length
    if len(password) < 8:
        errors.append("Password must be at least 8 characters long.")

    # Rule 2: Special character requirement
    if not re.search(r"[!@#$%^&*(),.?\":{}|<>]", password):
        errors.append("Password must contain at least one special character.")

    # Validation result
    if errors:
        return False, errors
    else:
        return True, ["Password is valid."]


def main():
    password = input("Enter password: ")

    valid, messages = validate_password(password)

    print("\nValidation Result:")
    for msg in messages:
        print("-", msg)


if __name__ == "__main__":
    main()
```

Output:

```
C:\AI_Assign>git init
Initialized empty Git repository in C:/AI_Assign/.git/

C:\AI_Assign>git add .

C:\AI_Assign>git commit -m "Initial commit"
On branch master

Initial commit
```

```
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding> & C:/Users/pooji/Ap
al/Python/pythoncore-3.14-64/python.exe c:/Users/pooji/OneDrive/Desktop/AI_A
_coding/assignment-21/task-03.py
Enter password: 123456

Validation Result:
- Password must be at least 8 characters long.
- Password must contain at least one special character.
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

This project is a Password Validation module that checks whether a user's password follows basic security rules.
The program validates conditions such as minimum password length and the presence of at least one special character.
After modifying the code to add these rules, commit messages are created to describe the updates made to the project.
AI is used to generate clear, structured commit messages that explain the changes professionally.
By comparing AI-generated and human-written messages, developers can understand how good commit messages improve code history and collaboration.

Task-04:

Prompt:

Student A:

Create the initial Python quiz program with predefined questions and answers.

Allow users to attempt the quiz.

Student B (with AI assistance):

Add score calculation functionality.

Implement input validation for user answers.

Guide the collaboration process by:

Showing Git branching for each student's work

Writing meaningful commit messages

Merging branches into the main project and handling conflicts if needed

Code:

```python
# quiz.py
# Initial Quiz Program (Student A)
def run_quiz():
    questions = [
        {"question": "What is the capital of France?", "answer": "Paris"},
        {"question": "What is 2 + 2?", "answer": "4"},
        {"question": "What color do you get when you mix red and white?", "answer": "Pink"}
    ]

    print("Welcome to the Quiz!")
    score = 0

    for q in questions:
        user_answer = input(q["question"] + " ")
        if user_answer.strip().lower() == q["answer"].lower():
            print("Correct!")
            score += 1
        else:
            print("Wrong! The correct answer is:", q["answer"])

    print(f"Your final score is: {score}/{len(questions)}")
if __name__ == "__main__":
    run_quiz()
```

Output:

```
C:\AI_Assign>git init
Initialized empty Git repository in C:/AI_Assign/.git/

C:\AI_Assign>git add .

C:\AI_Assign>git commit -m "Initial commit"
On branch master

Initial commit
```

```
_coding/assignment-21/task-04.py
Welcome to the Quiz!
What is the capital of France? Paris
Correct!
What is 2 + 2? 4
Correct!
What color do you get when you mix red and white? Gold
Wrong! The correct answer is: Pink
Your final score is: 2/3
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

This project is a Quiz Management System developed collaboratively by two students using Git and AI assistance.
Student A builds the basic quiz application containing questions and answers that users can attempt.
Student B enhances the program by adding score calculation and validating user inputs with help from AI.
Both students work on separate Git branches to avoid interfering with each other's changes.
Their work is later merged into the main project, demonstrating teamwork, version control, and AI-supported collaborative development.